
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 5

Funções em C

Exercícios de Autorrevisão (5.1–5.7)

Resolvidos passo a passo, abordagem incremental

Capítulos 1 + 2 + 3 + 4 + 5

Construindo sobre os Capítulos Anteriores

No Capítulo 5, transformamos a forma como organizamos nossos programas. Construindo sobre os fundamentos dos Capítulos 1–4, incorporamos:

- **Funções definidas pelo programador** – a base da modularização
- **Protótipos de funções** – contratos que o compilador verifica
- **Cabeçalho e corpo de função** – estrutura sintática
- **Parâmetros e argumentos** – passagem de informações entre funções
- **Funções da biblioteca matemática** (<math.h>): `sqrt`, `exp`, `log`, `pow`, `sin`, `cos`, `tan`, `ceil`, `floor`, `fabs`, `fmod`
- **Geração de números aleatórios** (`rand`, `srand`)
- **Classes de armazenamento** (`auto`, `register`, `static`, `extern`)
- **Escopo de identificadores** (função, arquivo, bloco, protótipo)
- **Recursão** – funções que chamam a si mesmas

Boa prática de programação

Funções são a unidade fundamental de organização em programas C de qualquer porte. Domine os conceitos de *parâmetros*, *tipo de retorno*, *escopo* e *recursão* – eles são a base da programação modular e da reutilização de software.

Contents

1	Exercício 5.1 – Preenchimento de Lacunas	2
2	Exercício 5.2 – Escopos no programa	6
3	Exercício 5.3 – Testar funções da biblioteca matemática	7
4	Exercício 5.4 – Cabeçalhos de função	10
5	Exercício 5.5 – Protótipos correspondentes	10
6	Exercício 5.6 – Declarações com classes de armazenamento	11
7	Exercício 5.7 – Identificar e Corrigir Erros	12

1. Exercício 5.1 – Preenchimento de Lacunas

Item (a)

Resposta detalhada

Resposta: Um módulo de programa em C é chamado de **função**.
Da Seção 5.4: “As funções permitem a criação de um programa em módulos”.
Em C, toda atividade de programação é feita por meio de funções – a própria `main` é uma função. Funções são a unidade fundamental de modularização.

Item (b)

Resposta detalhada

Resposta: Uma função é invocada com uma **chamada de função**.
A chamada de função consiste no nome da função seguido de parênteses contendo os argumentos, por exemplo: `quadrado(x)`. O termo *invocar* é sinônimo de *chamar* (*calling*).

Item (c)

Resposta detalhada

Resposta: **variável local**
Variáveis declaradas no corpo de uma função (ou em sua lista de parâmetros) só são conhecidas dentro daquela função. Após o retorno da função, essas variáveis deixam de existir (a menos que sejam `static`).

Item (d)

Resposta detalhada

Resposta: O comando `return`
`return expressão`; encerra a execução da função chamada e *retorna* o valor da expressão à função chamadora. Existe também `return`; sem expressão (usado em funções `void`).

Item (e)

Resposta detalhada

Resposta: **void**
A palavra-chave `void` tem dois usos no cabeçalho de uma função:

- Como *tipo de retorno*: `void instrucoes(...)` indica que a função **não retorna** valor algum.
- Como *lista de parâmetros*: `int main( void )` indica que a função **não recebe** nenhum parâmetro.

Item (f)

Resposta detalhada

Resposta: escopo

O *escopo* de um identificador é a região do programa onde ele pode ser referenciado. Em C, há quatro tipos de escopo: de função, de arquivo, de bloco e de protótipo de função (Seção 5.12).

Item (g)

Resposta detalhada

Resposta: `return;`, `return expressão;` e encontrando a chave direita de fechamento da função

Existem três formas de retornar o controle:

- a) `return;` – usado em funções `void`
- b) `return expressão;` – usado em funções com retorno de valor
- c) Atingir a chave de fechamento `}` da função (apenas em `void`)

Item (h)

Resposta detalhada

Resposta: protótipo de função

O protótipo informa ao compilador o nome da função, o tipo de retorno e os tipos e ordem dos parâmetros, permitindo verificar consistência das chamadas. Exemplo:

```
1 int quadrado( int y ); /* prototipo */
```

Item (i)

Resposta detalhada

Resposta: `rand`

A função `rand` (de `<stdlib.h>`) retorna um inteiro pseudoaleatório entre 0 e `RAND_MAX` (no mínimo 32767). Para gerar valores em um intervalo $[a, b]$, usa-se: `a + rand() % (b - a + 1)`.

Item (j)

Resposta detalhada

Resposta: `srand`

A função `srand` (*seed random*) define a “semente” do gerador pseudoaleatório. Sem `srand`, `rand` sempre produzirá a mesma sequência. A prática típica é usar o tempo atual como semente: `srand(time(NULL))`.

Item (k)

Resposta detalhada

Resposta: `auto`, `register`, `extern`, `static`

Os quatro especificadores de classe de armazenamento:

- `auto` – duração automática (default para variáveis locais)
- `register` – sugere armazenamento em registrador da CPU
- `extern` – variável definida em outro arquivo
- `static` – duração estática (mantém valor entre chamadas)

Item (l)

Resposta detalhada

Resposta: `auto`

Variáveis declaradas em um bloco ou na lista de parâmetros têm classe `auto` por padrão. Elas são criadas ao entrar no bloco e destruídas ao sair.

Item (m)

Resposta detalhada

Resposta: `register`

`register` é uma *recomendação* ao compilador para armazenar a variável em um registrador da CPU, garantindo acesso mais rápido. Compiladores modernos geralmente otimizam isso automaticamente, e o uso explícito de `register` é raro hoje em dia.

Item (n)

Resposta detalhada

Resposta: `externa` (ou `global`)

Variáveis declaradas fora de qualquer função têm *escopo de arquivo* e são chamadas de *externas* ou *globais*. Elas existem durante toda a execução do programa.

Item (o)

Resposta detalhada

Resposta: `static`

Uma variável local declarada com `static` mantém seu valor entre chamadas sucessivas da função. É inicializada apenas uma vez (na primeira execução).

```
1 void contador( void ) {  
2     static int n = 0; /* inicializa apenas na 1a  
    chamada */
```

```
3     ++n;  
4     printf( "Chamada numero %d\n", n );  
5 }
```

Item (p)

Resposta detalhada

Resposta: escopo de função, escopo de arquivo, escopo de bloco e escopo de protótipo de função

- **Função:** apenas para rótulos (label:, usados com goto)
- **Arquivo:** identificadores fora de funções – visíveis no arquivo todo
- **Bloco:** identificadores dentro de { }, criados ao entrar e destruídos ao sair
- **Protótipo:** parâmetros nomeados em protótipos – visíveis apenas no protótipo (não conflitam com nomes externos)

Item (q)

Resposta detalhada

Resposta: recursiva

Uma função recursiva é aquela que chama a si mesma – direta ou indiretamente. A recursão é uma técnica poderosa para resolver problemas que podem ser definidos em termos de uma versão menor de si mesmos.

Item (r)

Resposta detalhada

Resposta: básico

Toda função recursiva tem dois componentes:

- Caso básico:** condição que termina a recursão (sem chamada recursiva)
- Etapa de recursão:** expressa o problema em termos de uma versão menor

Erro comum de programação

Uma função recursiva sem caso básico (ou com caso básico inalcançável) gera recursão infinita – causando o esgotamento da *pilha de execução* (*stack overflow*) e o encerramento abrupto do programa.

2. Exercício 5.2 – Escopos no programa

Enunciado 5.2

Para o programa do enunciado, indicar o escopo de: (a) variável `x` em `main`; (b) variável `y` em `cube`; (c) função `cube`; (d) função `main`; (e) protótipo de `cube`; (f) identificador `y` no protótipo de `cube`.

Resposta detalhada

- a) `x` em `main` → **escopo de bloco** (visível apenas dentro do corpo da função `main`).
- b) `y` em `cube` → **escopo de bloco** (parâmetro – equivalente a uma variável local da função).
- c) Função `cube` → **escopo de arquivo** (definida em nível externo, visível em todo o arquivo).
- d) Função `main` → **escopo de arquivo**.
- e) Protótipo de `cube` → **escopo de arquivo** (declarado em nível externo).
- f) Identificador `y` no protótipo → **escopo de protótipo de função** (visível apenas dentro do próprio protótipo; não conflita com nomes externos).

Boa prática de programação

Os *nomes* dos parâmetros em protótipos podem ser omitidos – o que importa para o compilador são os *tipos*. Por exemplo, `int cube(int);` é equivalente a `int cube(int y);`.

3. Exercício 5.3 – Testar funções da biblioteca matemática

Resposta detalhada

```
1  /* Ex5.3: testa_math.c
2     Testa funcoes da biblioteca matematica <math.h>
3     Compilar com: gcc ex05_03.c -o ex05_03 -lm */
4  #include <stdio.h>
5  #include <math.h>
6
7  int main( void )
8  {
9     /* Raiz quadrada */
10    printf( "sqrt(%.1f) = %.1f\n", 900.0, sqrt( 900.0
11        ) );
12    printf( "sqrt(%.1f) = %.1f\n", 9.0,    sqrt( 9.0 )
13        );
14
15    /* Funcao exponencial e^x */
16    printf( "exp(%.1f) = %f\n", 1.0, exp( 1.0 ) );
17    printf( "exp(%.1f) = %f\n", 2.0, exp( 2.0 ) );
18
19    /* Logaritmo natural (base e) */
20    printf( "log(%f) = %.1f\n", 2.718282, log(
21        2.718282 ) );
22    printf( "log(%f) = %.1f\n", 7.389056, log(
23        7.389056 ) );
24
25    /* Logaritmo (base 10) */
26    printf( "log10(%.1f) = %.1f\n", 1.0,    log10( 1.0
27        ) );
28    printf( "log10(%.1f) = %.1f\n", 10.0,   log10( 10.0
29        ) );
30    printf( "log10(%.1f) = %.1f\n", 100.0, log10(
31        100.0 ) );
32
33    /* Valor absoluto */
34    printf( "fabs(%.1f) = %.1f\n", 13.5, fabs( 13.5 )
35        );
36    printf( "fabs(%.1f) = %.1f\n", 0.0, fabs( 0.0 )
37        );
38    printf( "fabs(%.1f) = %.1f\n", -13.5, fabs( -13.5
39        ) );
40
41    /* Teto e piso */
42    printf( "ceil(%.1f) = %.1f\n", 9.2, ceil( 9.2 ) )
43        ;
44    printf( "ceil(%.1f) = %.1f\n", -9.8, ceil( -9.8 )
45        );
46    printf( "floor(%.1f) = %.1f\n", 9.2, floor( 9.2 )
```



```

    );
35 printf( "floor(%.1f) = %.1f\n", -9.8, floor( -9.8
    ) );
36
37 /* Potencia */
38 printf( "pow(%.1f, %.1f) = %.1f\n", 2.0, 7.0, pow(
    2.0, 7.0 ) );
39 printf( "pow(%.1f, %.1f) = %.1f\n", 9.0, 0.5, pow(
    9.0, 0.5 ) );
40
41 /* Modulo de ponto flutuante */
42 printf( "fmod(%.3f/%.3f) = %.3f\n",
43         13.675, 2.333, fmod( 13.675, 2.333 ) );
44
45 /* Funcoes trigonometricas (radianos) */
46 printf( "sin(%.1f) = %.1f\n", 0.0, sin( 0.0 ) );
47 printf( "cos(%.1f) = %.1f\n", 0.0, cos( 0.0 ) );
48 printf( "tan(%.1f) = %.1f\n", 0.0, tan( 0.0 ) );
49
50 return 0;
51 } /* fim da funcao main */

```

Saída do programa

```

sqrt(900.0) = 30.0
sqrt(9.0) = 3.0
exp(1.0) = 2.718282
exp(2.0) = 7.389056
log(2.718282) = 1.0
log(7.389056) = 2.0
log10(1.0) = 0.0
log10(10.0) = 1.0
log10(100.0) = 2.0
fabs(13.5) = 13.5
fabs(0.0) = 0.0
fabs(-13.5) = 13.5
ceil(9.2) = 10.0
ceil(-9.8) = -9.0
floor(9.2) = 9.0
floor(-9.8) = -10.0
pow(2.0, 7.0) = 128.0
pow(9.0, 0.5) = 3.0
fmod(13.675/2.333) = 2.010
sin(0.0) = 0.0
cos(0.0) = 1.0
tan(0.0) = 0.0

```

Boa prática de programação

Em sistemas Linux/macOS, ao usar funções de `<math.h>`, é necessário ligar a biblioteca matemática durante a compilação: `gcc programa.c -o programa -lm`. Sem o `-lm`, o linker reportará “referência indefinida” para funções como `sqrt`, `pow`, etc.

4. Exercício 5.4 – Cabeçalhos de função

Resposta detalhada

a) Função `hypotenuse` – 2 argumentos `double`, retorna `double`:

```
1 double hypotenuse( double side1, double side2 )
```

b) Função `smallest` – 3 `int`, retorna `int`:

```
1 int smallest( int x, int y, int z )
```

c) Função `instructions` – nada e nada:

```
1 void instructions( void )
```

d) Função `intToFloat` – 1 `int`, retorna `float`:

```
1 float intToFloat( int number )
```

Observação importante (Erro Comum 5.3): ao declarar múltiplos parâmetros do mesmo tipo, é **obrigatório** repetir o tipo para cada um. Escrever `double side1, side2` é *erro de compilação* – deve ser `double side1, double side2`.

5. Exercício 5.5 – Protótipos correspondentes

Resposta detalhada

Os protótipos são idênticos aos cabeçalhos do Exercício 5.4, terminando com `;` (em vez de `{...}`):

```
1 double hypotenuse( double side1, double side2 );
2 int smallest( int x, int y, int z );
3 void instructions( void );
4 float intToFloat( int number );
```

Boa prática de programação

Os protótipos podem omitir os nomes dos parâmetros, deixando apenas os tipos: `double hypotenuse( double, double );`. O compilador só precisa dos tipos para verificar a consistência das chamadas.

6. Exercício 5.6 – Declarações com classes de armazenamento

Resposta detalhada

a) Inteiro count em registrador, inicializado em 0:

```
1 register int count = 0;
```

b) Variável lastVal (float) que retém valor entre chamadas:

```
1 static float lastVal;
```

c) Inteiro externo number restrito ao arquivo:

```
1 static int number; /* fora de qualquer funcao */
```

Nota: no contexto de variáveis externas (declaradas fora de funções), **static** restringe o escopo ao arquivo onde foi definida – isto é, outros arquivos do projeto não poderão acessá-la (mesmo com **extern**). Isso é uma forma de *ocultação de informações* no nível de arquivos.

7. Exercício 5.7 – Identificar e Corrigir Erros

Item (a) – Função aninhada

Resposta detalhada

Erro: A função `h` está definida *dentro* da função `g`. Em C, **não é permitido** definir uma função dentro de outra.

Correção: mover a definição de `h` para fora de `g`.

```
1  int g( void )
2  {
3      printf( "Dentro da funcao g\n" );
4  }
5
6  int h( void )      /* movida para fora */
7  {
8      printf( "Dentro da funcao h\n" );
9  }
```

Item (b) – Falta retorno

Resposta detalhada

Erro: A função `sum` é declarada como `int` (deve retornar um inteiro), mas *não há return*. O resultado é indefinido.

Correção:

```
1  int sum( int x, int y )
2  {
3      return x + y;  /* simplifica eliminando 'result'
4                      */
5  }
```

Item (c) – Recursão sem retorno + chave faltando

Resposta detalhada

Erros: (1) Falta `}` antes do `else` (sintaxe); (2) na cláusula `else`, `n + sum(n - 1)` é calculado mas *não retornado*.

Correção:

```
1  int sum( int n )
2  {
3      if ( n == 0 ) {
4          return 0;
5      }
6      else {
7          return n + sum( n - 1 );  /* adiciona return
```

```

8         */
9     }

```

Item (d) – ; após cabeçalho + redefinição de parâmetro

Resposta detalhada

Erros: (1) Há ; após void f(float a) – isso transforma o “cabeçalho” em um protótipo, e o que vem depois fica órfão; (2) O parâmetro a é *redeclarado* dentro do corpo (float a;), o que é erro de compilação.

Correção:

```

1 void f( float a )      /* sem ; */
2 {
3     /* sem 'float a;' aqui */
4     printf( "%f", a );
5 }

```

Item (e) – return em função void

Resposta detalhada

Erro: A função product é declarada como void, mas tenta retornar um valor (return result;). Isso é **erro de compilação** (Erro Comum 5.2).

Correção: simplesmente remover a instrução return result;.

```

1 void product( void )
2 {
3     int a, b, c, result;
4     printf( "Digite tres inteiros: " );
5     scanf( "%d%d%d", &a, &b, &c );
6     result = a * b * c;
7     printf( "Resultado e %d", result );
8     /* sem return aqui */
9 }

```

Alternativa: se realmente quisermos retornar o resultado, mudamos o tipo:

```

1 int product( void )      /* int em vez de void */
2 {
3     /* ... */
4     return result;      /* agora valido */
5 }

```

Gabarito Rápido – Capítulo 5: Autorrevisão

Respostas Resumidas

5.1:

- a) função b) chamada de função c) variável local
- d) `return` e) `void` f) escopo
- g) `return;`, `return expr`, ou chave de fechamento
- h) protótipo de função i) `rand` j) `srand`
- k) `auto`, `register`, `extern`, `static`
- l) `auto` m) `register` n) externa/global o) `static`
- p) função, arquivo, bloco, protótipo q) recursiva r) básico

5.2: Todas as variáveis dentro de `main` ou `cube` têm escopo de **bloco**. As funções e o protótipo têm escopo de **arquivo**. O nome `y` no protótipo tem escopo de **protótipo**.

5.4 e 5.5: Cabeçalhos completos com `double`, `int`, `void`, `float`.

5.6: (a) `register int count = 0;` (b) `static float lastVal;` (c) `static int number;`

5.7: 5 erros corrigidos: função aninhada, falta de `return`, sintaxe, ponto e vírgula extra, `return` em `void`.